

ASSIGNMENT 11.2

Hemavathi N

2303A51965

Batch 24

AIAC

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

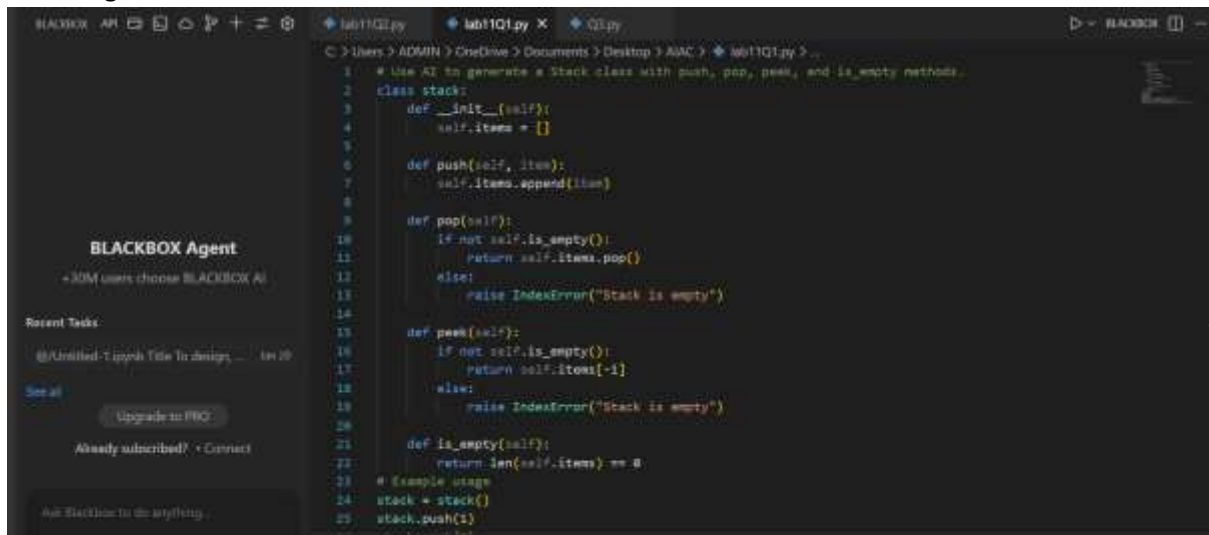
Sample Input Code:

class Stack:

pass

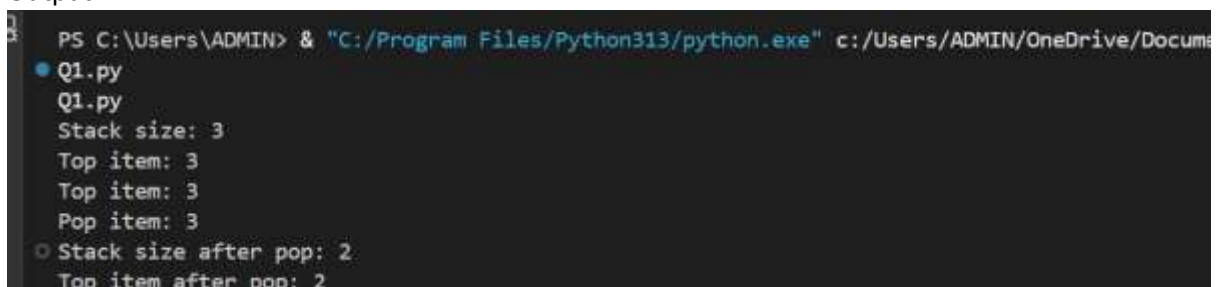
Expected Output:

- A functional stack implementation with all required methods and docstrings.



The screenshot shows a code editor with a Python file named 'lab11Q1.py'. The code defines a 'Stack' class with the following methods:
- `__init__(self)`: Initializes an empty list 'self.items'.
- `push(self, item)`: Appends the item to the stack.
- `pop(self)`: Removes and returns the top item, raising an `IndexError` if the stack is empty.
- `peek(self)`: Returns the top item without removing it, raising an `IndexError` if the stack is empty.
- `is_empty(self)`: Returns a boolean indicating if the stack is empty.
Example usage is shown at the bottom: `s = Stack()`, `s.push(1)`, `s.push(2)`, `s.push(3)`, `s.pop()`, `s.pop()`, `s.pop()`, `s.peek()`, `s.is_empty()`.

Output :



The terminal output shows the execution of the Python script:
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Docume
Q1.py
Q1.py
Stack size: 3
Top item: 3
Top item: 3
Pop item: 3
Stack size after pop: 2
Top item after pop: 2

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

class Queue:

pass

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size

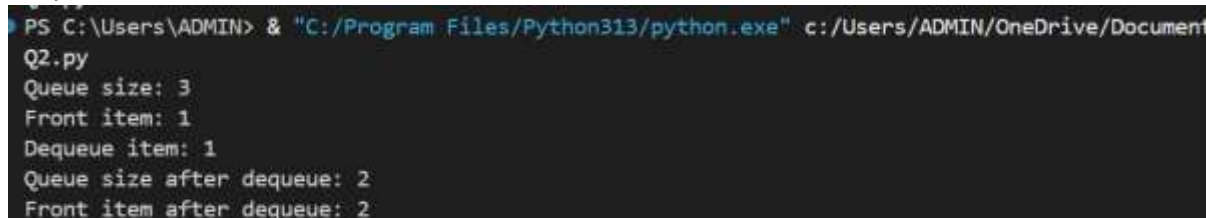
methods.



The screenshot shows a code editor with a file named `lab11Q2.py`. The code implements a `Queue` class with the following methods: `__init__`, `is_empty`, `enqueue`, `dequeue`, `peek`, and `size`. An example usage is provided at the bottom of the code block.

```
1 # Write a code to implement queue and demonstrate its operations
2 class Queue:
3     def __init__(self):
4         self.items = []
5     def is_empty(self):
6         return len(self.items) == 0
7     def enqueue(self, item):
8         self.items.append(item)
9     def dequeue(self):
10        if not self.is_empty():
11            return self.items.pop(0)
12        else:
13            raise IndexError("Queue is empty")
14    def peek(self):
15        if not self.is_empty():
16            return self.items[0]
17        else:
18            raise IndexError("Queue is empty")
19    def size(self):
20        return len(self.items)
21
22 # Example usage
23 queue = Queue()
24 queue.enqueue(1)
```

Output :



The terminal window shows the execution of the `Queue` program. The output is as follows:

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Document
Q2.py
Queue size: 3
Front item: 1
Dequeue item: 1
Queue size after dequeue: 2
Front item after dequeue: 2
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

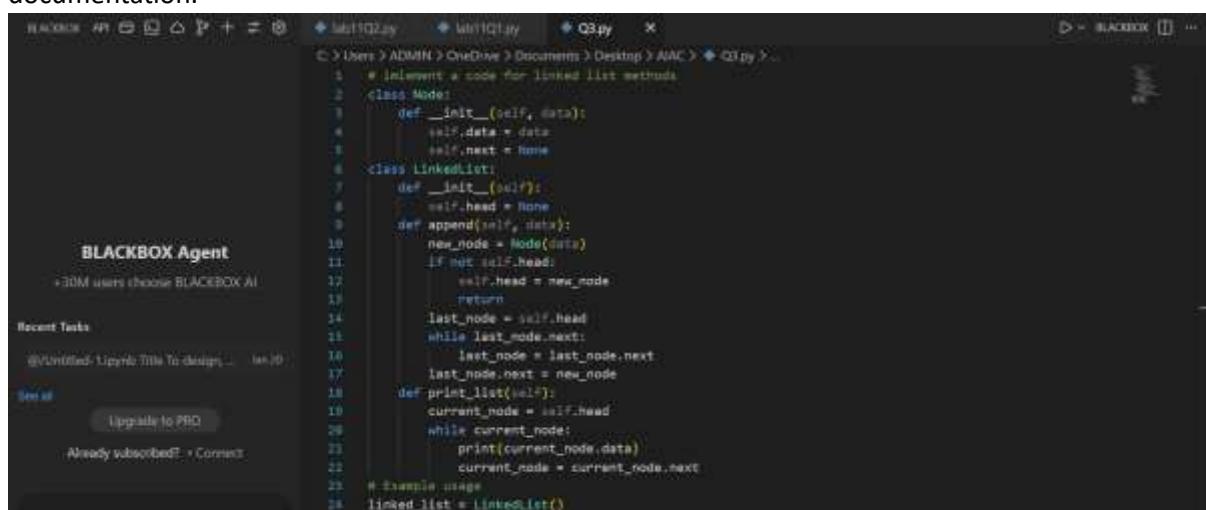
pass

class LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation.



The screenshot shows a code editor with a file named `Q3.py`. The code implements a Singly Linked List with the following classes and methods: `Node` (a simple class), `LinkedList` (with methods `__init__`, `append`, `print_list`), and an example usage at the bottom.

```
1 # Implement a code for linked list methods
2 class Node:
3     def __init__(self, data):
4         self.data = data
5         self.next = None
6
7 class LinkedList:
8     def __init__(self):
9         self.head = None
10    def append(self, data):
11        new_node = Node(data)
12        if not self.head:
13            self.head = new_node
14            return
15        last_node = self.head
16        while last_node.next:
17            last_node = last_node.next
18        last_node.next = new_node
19    def print_list(self):
20        current_node = self.head
21        while current_node:
22            print(current_node.data)
23            current_node = current_node.next
24
25 # Example usage
26 linked_list = LinkedList()
```

Output :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/AIAC/Q3.py
1
2
3
```

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.

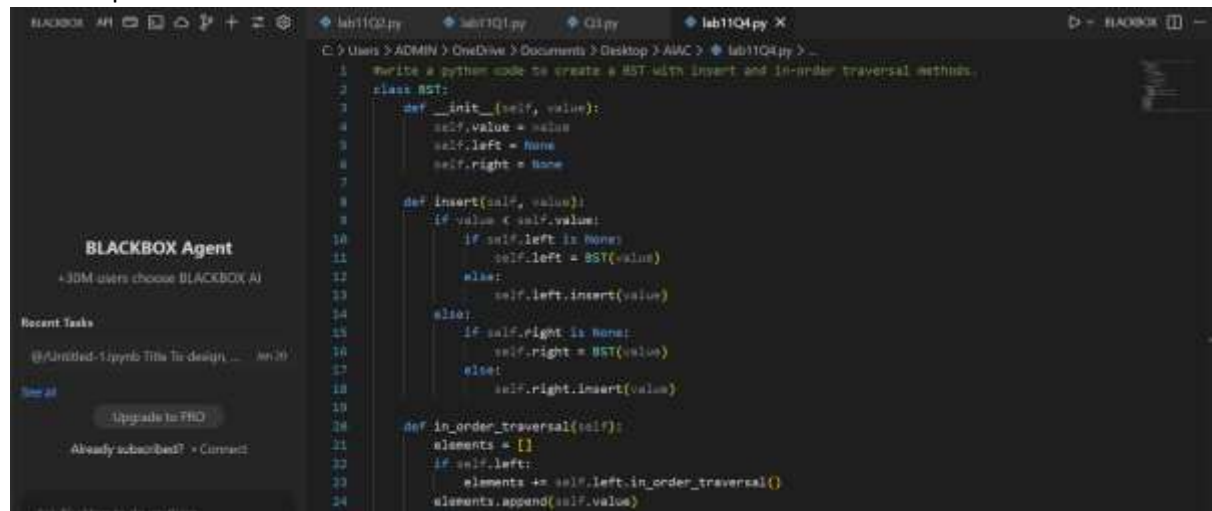
Sample Input Code:

class BST:

pass

Expected Output:

- BST implementation with recursive insert and traversal methods.



```
class BST:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

    def insert(self, value):
        if value < self.value:
            if self.left is None:
                self.left = BST(value)
            else:
                self.left.insert(value)
        else:
            if self.right is None:
                self.right = BST(value)
            else:
                self.right.insert(value)

    def in_order_traversal(self):
        elements = []
        if self.left:
            elements += self.left.in_order_traversal()
        elements.append(self.value)
        if self.right:
            elements += self.right.in_order_traversal()
        return elements
```

Output :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/Q4.py
In-order Traversal: [3, 5, 7, 10, 12, 15, 18]
PS C:\Users\ADMIN>
```

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:

pass

Expected Output:

- Collision handling using chaining, with well-commented methods.

```

> Users > ADMIN > OneDrive > Documents > Desktop > AIAC > lab11Q5.py > ...
1  #write a python code to implement a hash table with basic insert, search, and delete methods Collision
2  class HashTable:
3      def __init__(self, size=10):
4          self.size = size
5          self.table = [[] for _ in range(size)] # Create a list of empty lists for chaining
6
7      def _hash(self, key):
8          """Generate a hash for the given key."""
9          return hash(key) % self.size
10
11     def insert(self, key, value):
12         """Insert a key-value pair into the hash table."""
13         index = self._hash(key)
14         # Check if the key already exists and update it
15         for i, (k, v) in enumerate(self.table[index]):
16             if k == key:
17                 self.table[index][i] = (key, value) # Update existing key
18                 return
19         # If the key does not exist, add a new key-value pair
20         self.table[index].append((key, value))
21
22     def search(self, key):
23         """Search for a value by its key."""
24         index = self._hash(key)
25         for k, v in self.table[index]:

```

Output :

```

PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.
● Q5.py
  Q5.py
  Name: Alice
○ Age: 30

```

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

```
class Graph:
```

```
pass
```

Expected Output:

- Graph with methods to add vertices, add edges, and display connections.

```

C:\Users\ADMIN\OneDrive\Documents\Desktop\AIAC> lab11Q6.py > ...
1  #Write a python code to implement a graph using an adjacency list Graph with methods to add vertices,
2  class Graph:
3      def __init__(self):
4          self.graph = {}
5
6      def add_vertex(self, vertex):
7          if vertex not in self.graph:
8              self.graph[vertex] = []
9
10     def add_edge(self, vertex1, vertex2):
11         if vertex1 in self.graph and vertex2 in self.graph:
12             self.graph[vertex1].append(vertex2)
13             self.graph[vertex2].append(vertex1)
14
15     def display_connections(self):
16         for vertex, edges in self.graph.items():
17             print(f'{vertex}: {', '.join(edges)}')
18
19     # Example usage
20     graph = Graph()
21     graph.add_vertex("A")
22     graph.add_vertex("B")
23     graph.add_vertex("C")
24     graph.add_edge("A", "B")
25     graph.add_edge("A", "C")
26     graph.display_connections()

```

Output :

```

C:\Users\ADMIN> python C:\Program Files\Python315\python.exe C:\Users\ADMIN\
Q6.py
A: B, C
B: A
C: A
B: A
C: A
C: A

```

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

```

class PriorityQueue:
    pass

```

Expected Output:

- Implementation with enqueue (priority), dequeue (highest priority), and display methods.

```
# Write a python code to implement a priority queue using Python's heapq module Implementation with :
class PriorityQueue:
    def __init__(self):
        self.items = []

    def is_empty(self):
        return len(self.items) == 0

    def enqueue(self, priority, item):
        self.items.append((priority, item))
        self.items.sort(key=lambda x: x[0]) # Sort by priority (lowest first)

    def dequeue(self):
        if not self.is_empty():
            return self.items.pop(0)[1] # Return the item with the highest priority
        else:
            raise IndexError("Queue is empty")

    def peek(self):
        if not self.is_empty():
            return self.items[0][1] # Return the item with the highest priority without removing it
        else:
            raise IndexError("Queue is empty")

    def size(self):
        return len(self.items)

# Example usage
```

Output :

```
> ▾ TERMINAL
PS C:\Users\ADMIN> ...
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/Q7.py
Queue size: 3
Front item: Task 1
Dequeue item: Task 1
Queue size after dequeue: 2
Front item after dequeue: Task 2
PS C:\Users\ADMIN> 
```

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using `collections.deque`.

Sample Input Code:

```
class DequeDS:
```

```
pass
```

Expected Output:

- Insert and remove from both ends with docstrings.

```
Q3.py lab11Q4.py lab11Q5.py lab11Q6.py lab11Q7.py lab11Q8.py BLACKBOX
C:\Users\ADMIN> OneDrive\Documents\Desktop\AIAC\lab11Q8.py
1 #write a python code to implement a double-ended queue using collections deque.Insert and remove from
2 from collections import deque
3 class DequeDS:
4     def __init__(self):
5         self.items = deque()
6
7     def is_empty(self):
8         return len(self.items) == 0
9
10    def insert_front(self, item):
11        """Insert an item at the front of the deque."""
12        self.items.appendleft(item)
13
14    def insert_rear(self, item):
15        """Insert an item at the rear of the deque."""
16        self.items.append(item)
17
18    def remove_front(self):
19        """Remove and return the item from the front of the deque."""
20        if not self.is_empty():
21            return self.items.popleft()
22        else:
23            raise IndexError("Deque is empty")
24
25    def remove_rear(self):
```

Output :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/
Q8.py
Front item: 0
Rear item: 2
Deque size: 3
Remove front item: 0
Remove rear item: 2
Deque size after removals: 1
```

Task Description #9 Real-Time Application Challenge – Choose the Right Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue

- o Priority Queue
- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
C > Users > ADMIN > OneDrive > Documents > Desktop > AIAC > lab11Q9.py > ...
1  # write a python program to display Traffic Signal Queue - Vehicles waiting at signals
2  import time
3  import random
4  class TrafficSignal:
5      def __init__(self):
6          self.vehicles_queue = []
7      def add_vehicle(self, vehicle):
8          self.vehicles_queue.append(vehicle)
9      def display_queue(self):
10         print("Vehicles waiting at the signal:")
11         for vehicle in self.vehicles_queue:
12             print(vehicle)
13     def change_signal(self):
14         if self.vehicles_queue:
15             print("Signal is green. Vehicles can go.")
16             self.vehicles_queue.clear()
17         else:
18             print("Signal is red. No vehicles waiting.")
19     def main():
20         traffic_signal = TrafficSignal()
21         vehicle_types = ['Car', 'Bus', 'Motorcycle', 'Truck']
22         for _ in range(10):
```

Output :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/AIAC/lab11
Q9.py
Vehicles waiting at the signal:
Bus
Motorcycle
Motorcycle
Car
Bus
Car
Truck
Truck
Car
Motorcycle
Signal is green. Vehicles can go.
```

Task Description #10: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.

2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
> Users > ADMIN > OneDrive > Documents > Desktop > AIAC > lab11Q10.py > ...
1  # write a python program to manage an emergency room patient queue based on triage priority using a #
2  import heapq
3  class EmergencyRoom:
4      """
5      A system to manage emergency patients based on triage priority.
6      Priority 1 is the most urgent (Critical), while higher numbers are less urgent.
7      """
8      def __init__(self):
9          # The heap stores tuples: (priority_level, patient_name)
10         self.patient_heap = []
11
12     def admit_patient(self, name, priority):
13         """
14         Adds a patient to the emergency queue.
15         :param name: Name of the patient.
16         :param priority: Integer (1 = Critical, 2 = Serious, 3 = Stable).
17         """
18         heapq.heappush(self.patient_heap, (priority, name))
19         print(f"Admitted: {name} (Priority Level: {priority})")
20
21     def treat_next_patient(self):
22         """Removes and returns the patient with the highest priority."""
23         if not self.patient_heap:
24             print("No patients in the waiting area.")
25             return None
```

Output :

```
PS C:\Users\ADMIN> & C:/Program Files/Python313/python.exe C:/Users/ADMIN/OneDrive/Documents/Desktop/Q10.py
Admitted: John Doe (Priority Level: 3)
Admitted: Jane Smith (Priority Level: 1)
Admitted: Baby Sam (Priority Level: 2)

--- Current Emergency Waiting List ---
Priority 1: Jane Smith
Priority 2: Baby Sam
Priority 3: John Doe
-----

>>> Treating Patient: Jane Smith [Status: CRITICAL]
```

Task Description #11: Smart City Traffic Control System

A city plans a Smart Traffic Management System that includes:

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

C:\Users > ADMIN > OneDrive > Documents > Desktop > AIAC > lab11Q11.py > ...
1  # write a python program to manage an emergency room patient queue based on triage priority using a F
2  import heapq
3  class EmergencyRoom:
4      """
5      A system to manage emergency patients based on triage priority.
6      Priority 1 is the most urgent (Critical), while higher numbers are less urgent.
7      """
8      def __init__(self):
9          # The heap stores tuples: (priority_level, patient_name)
10         self.patient_heap = []
11
12     def admit_patient(self, name, priority):
13         """
14         Adds a patient to the emergency queue.
15         :param name: Name of the patient.
16         :param priority: Integer (1 = Critical, 2 = Serious, 3 = Stable).
17         """
18         heapq.heappush(self.patient_heap, (priority, name))
19         print(f"Admitted: {name} (Priority Level: {priority})")
20     def treat_next_patient(self):
21         """Removes and returns the patient with the highest priority."""
22         if not self.patient_heap:
23             print("No patients in the waiting area.")
24             return None

```

Output :

```

Q11.py
Admitted: John Doe (Priority Level: 3)
Admitted: Jane Smith (Priority Level: 1)
Admitted: Baby Sam (Priority Level: 2)

--- Current Emergency Waiting List ---
Priority 1: Jane Smith
Priority 2: Baby Sam
Priority 3: John Doe
-----

>>> Treating Patient: Jane Smith [Status: CRITICAL]
>>> Treating Patient: Baby Sam [Status: SERIOUS]

```

Task Description #12: Smart E-Commerce Platform – Data Structure Challenge

An e-commerce company wants to build a Smart Online Shopping System with:

1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue

- o Linked List
- o Binary Search Tree (BST)
- o Graph
- o Hash Table
- o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
C:\Users\ADMIN> OneDrive\Documents\Desktop\AIAC> lab11Q12.py > ...
1  #Implement one selected feature as a working Python program
2  from collections import deque
3  class OrderQueue:
4      """
5      Manage e-commerce orders using FIFO (First-In-First-Out) queue.
6      """
7      def __init__(self):
8          """Initialize an empty order queue."""
9          self.orders = deque()
10     def place_order(self, order_id):
11         """Add a new order to the queue."""
12         self.orders.append(order_id)
13         print(f"Order {order_id} placed.")
14     def process_order(self):
15         """Process and remove the next order in the queue."""
16         if self.orders:
17             order = self.orders.popleft()
18             print(f"Processing order {order}")
19             return order
20         print("No orders to process.")
21         return None
22     def display_orders(self):
23         """Display all pending orders."""
24         print("Pending Orders:", list(self.orders))
25 # Example Usage
```

Output :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Docu
Q12.py
Order ORD101 placed.
Order ORD102 placed.
Order ORD103 placed.
Pending Orders: ['ORD101', 'ORD102', 'ORD103']
Processing order ORD101
Pending Orders: ['ORD102', 'ORD103']
```

Task #13: Smart Logistics & Warehouse Management

Scenario:

A logistics company wants to implement:

1. Shipment Processing
2. Urgent Shipment Priority
3. Product Lookup by Code
4. Warehouse Network Connectivity
5. Inventory Add/Remove Operations

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
 - Justify your choice in 2–3 sentences per feature.
 - Implement one selected feature as a working Python program with AI-assisted code generation.
- Expected Output:
- A table mapping feature → chosen data structure → justification.
 - A functional Python program implementing the chosen feature with comments and docstrings.

```

C:\Users\ADMIN> OneDrive\Documents\Desktop\AIAC> lab11Q13.py > _
13 class UrgentShipmentQueue:
14     """
15     Manage shipments in a warehouse using a Priority Queue.
16     Urgent shipments with higher priority are processed first.
17     """
18     def __init__(self):
19         """Initialize an empty priority queue."""
20         self.shipments = []
21     def add_shipment(self, shipment_id, priority):
22         """
23         Add a shipment to the priority queue.
24         Args:
25             shipment_id (str): Unique ID of the shipment
26             priority (int): Priority of the shipment (1 = highest)
27         """
28         heapq.heappush(self.shipments, (priority, shipment_id))
29         print(f"Shipment {shipment_id} added with priority {priority}")
30     def process_shipment(self):
31         """
32         Process the highest priority shipment.
33         Returns:
34             str: Shipment ID that was processed
35         """
36         if self.shipments:

```

Output :

```

PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDr
Q13.py
Shipment SHIP1001 added with priority 2
Shipment SHIP1002 added with priority 1
Shipment SHIP1003 added with priority 3
Current Shipment Queue: [(1, 'SHIP1002'), (2, 'SHIP1001'), (3, 'SHIP1003')]
Processing shipment SHIP1002 (Priority 1)
Current Shipment Queue: [(2, 'SHIP1001'), (3, 'SHIP1003')]

```

Task Description #14: Smart Banking Management System – Data Structure Challenge

A bank plans to develop a Smart Banking System that includes:

1. Customer Service Token System – Customers are served in order of arrival.

2. Loan Approval Priority Handling – High-value or urgent loans processed first.
3. Account Information Lookup – Fast retrieval of customer details using Account Number.
4. Transaction History Management – Maintain chronological transaction records.
5. ATM Network Connectivity – Represent connections between ATMs across cities.

Student Task:

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
C:\Users\ADMIN> OneDrive\Documents\Desktop\AIAC> lab11Q14.py > _
1  # write a python program to display a bank registry system using a hash table to store account info
2  class BankRegistry:
3      """
4      A system to manage bank account records using a Hash Table.
5      Provides O(1) average time complexity for lookups and updates.
6      """
7      def __init__(self):
8          # Dictionary acts as the Hash Table: Key=AccountNum, Value=Details
9          self.accounts = {}
10
11      def create_account(self, account_num, name, balance):
12          """Adds a new account to the registry."""
13          if account_num in self.accounts:
14              print(f"Error: Account {account_num} already exists.")
15          else:
16              self.accounts[account_num] = {
17                  "name": name,
18                  "balance": balance,
19                  "status": "Active"
20              }
21              print(f"Account created successfully for {name}.")
22
23      def get_account_details(self, account_num):
```

Output :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/AI
Q14.py
Account created successfully for Alice Smith.
Account created successfully for Bob Jones.
Account created successfully for Charlie Day.

--- Account Statement: AC-1001 ---
Holder: Alice Smith
Balance: $5,000.00
Status: Active
```

Task Description #15: Online Learning Management System – Data Structure Selection

An online education platform wants to develop a Learning Management System that handles:

1. Student Login Authentication – Quick validation of user credentials.
2. Assignment Submission Queue – Submissions processed in order of upload.
3. Top Performer Ranking – Rank students based on scores.
4. Course Prerequisite Structure – Represent subject dependencies.
5. Undo Feature in Online Editor – Reverse recent actions.

Student Task:

- Select the most appropriate data structure for each feature from the given list.
- Justify your selection in 2–3 sentences per feature.
- Implement one selected feature in Python using AI-assisted code generation.

Expected Output:

- Feature → Data Structure → Justification table.
- Functional Python program with proper documentation.

```
C > Users > ADMIN > OneDrive > Documents > Desktop > AIAC > lab11Q15.py > ...
1  #Write a python program to display Assignment Submission Queue – Submissions processed in order of up
2  import heapq
3  class AssignmentSubmissionQueue:
4      def __init__(self):
5          self.submission_queue = []
6          self.counter = 0 # To maintain the order of submissions
7
8      def submit_assignment(self, student_name, assignment_name):
9          # Use a tuple (counter, student_name, assignment_name) to maintain order
10         heapq.heappush(self.submission_queue, (self.counter, student_name, assignment_name))
11         self.counter += 1
12         print(f"Assignment '{assignment_name}' submitted by {student_name}.")
13
14     def process_submission(self):
15         if self.submission_queue:
16             submission = heapq.heappop(self.submission_queue)
17             print(f"Processing submission: {submission[1]} - {submission[2]}")
18         else:
19             print("No submissions to process.")
20
21     # Example usage
22     if __name__ == "__main__":
23         submission_queue = AssignmentSubmissionQueue()
24         submission_queue.submit_assignment("Alice", "Math Homework")
25
26 Chat (CTRL + I) / Share (CTRL + S)
```

Output :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/AIAC/1Q15.py
Assignment 'Science Project' submitted by Bob.
Assignment 'History Essay' submitted by Charlie.
Processing submission: Alice - Math Homework
Processing submission: Bob - Science Project
Processing submission: Charlie - History Essay
Processing submission: Bob - Science Project
Processing submission: Bob - Science Project
```

Task Description #16: Smart Airport Management System – Data Structure Challenge

An airport authority plans to implement:

1. Passenger Check-In Queue – Process passengers in order of arrival.
2. Emergency Landing Priority – Aircraft in emergency get priority.
3. Flight Information Lookup – Fast search using Flight ID.
4. Air Route Connectivity – Airports connected via routes.
5. Boarding Process (Last-In First-Out Cabin Storage) – Overhead luggage management simulation.

Student Task:

- Choose suitable data structures from the provided list.
- Provide 2–3 sentence justification per feature.
- Implement one feature in Python with comments and docstrings using AI assistance.

Expected Output:

- Mapping table.
- Working Python implementation.

Output :

```
> Users > ADMIN > OneDrive > Documents > Desktop > AIAC > lab11Q16.py > ...
1  #write a python program that simulates an air traffic control system managing aircraft landing sequen
2  import heapq
3  class AirTrafficControl:
4      """
5      Manages aircraft landing sequences based on priority levels.
6      Priority 1: Emergency (Critical)
7      Priority 2: Low Fuel (Urgent)
8      Priority 3: Scheduled (Normal)
9      """
10     def __init__(self):
11         # Min-heap stores tuples: (priority_level, flight_id)
12         self.landing_queue = []
13     def request_landing(self, flight_id, priority):
14         """
15         Adds a flight to the landing sequence with a specific priority.
16         :param flight_id: Unique identifier for the aircraft (e.g., 'DAL452').
17         :param priority: Integer priority level (1=Highest, 3=Lowest).
18         """
19         heapq.heappush(self.landing_queue, (priority, flight_id))
20         status = {1: "EMERGENCY", 2: "LOW FUEL", 3: "SCHEDULED"}.get(priority, "NORMAL")
21         print(f"Inbound: {flight_id} | Status: {status} | Priority Level: {priority}")
22     def clear_for_landing(self):
23         """Clears the highest-priority aircraft for landing."""
24         if not self.landing_queue:
```

Output :

```
Q16.py
Inbound: UAE202 | Status: SCHEDULED | Priority Level: 3
Inbound: BAW117 | Status: SCHEDULED | Priority Level: 3
Inbound: AFR088 | Status: EMERGENCY | Priority Level: 1
Inbound: DLH400 | Status: LOW FUEL | Priority Level: 2

--- Current Landing Sequence ---
[EMERGENCY] Flight: AFR088
[LOW FUEL] Flight: DLH400
```

Task Description #17: Smart Social Media Platform – Data Structure Selection

A social media company wants to develop a system that includes:

1. News Feed Generation – Display posts chronologically.
2. Trending Hashtags Tracker – Rank hashtags by usage frequency.
3. User Profile Search – Quick lookup using username.
4. Friend Network Representation – Represent user connections.
5. Message Undo Feature – Allow last sent message to be withdrawn.

Student Task:

- Select appropriate data structures from the list.
- Justify your choices clearly.
- Implement one selected feature as a Python program using AI-assisted code generation.

Expected Output:

- Table mapping features to data structures with justification.
- Working Python code with docstrings and comments.

```
1 #write a python program that simulates a social media platform's trending topics feature.
2 import heapq
3 class TrendingTracker:
4     """
5     Tracks hashtag frequency and retrieves top trending topics
6     using a Priority Queue (Max-Heap simulation).
7     """
8     def __init__(self):
9         # Dictionary to keep track of hashtag counts: {hashtag: count}
10        self.hashtag_counts = {}
11
12    def add_tag(self, hashtag):
13        """Increments the usage count of a specific hashtag."""
14        hashtag = hashtag.lower().strip("#")
15        self.hashtag_counts[hashtag] = self.hashtag_counts.get(hashtag, 0) + 1
16        print(f"Update: #{hashtag} is being talked about!")
17
18    def get_trending(self, n=3):
19        """
20        Retrieves the top N trending hashtags.
21        Uses a heap to find the largest elements efficiently.
22        """
23        if not self.hashtag_counts:
```

Output :



```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/AIAC/lab11/Q17.py
Update: #python is being talked about!
Update: #ai is being talked about!
Update: #coding is being talked about!
Update: #ai is being talked about!
Update: #tech2026 is being talked about!
Update: #ai is being talked about!
Update: #python is being talked about!

--- Top 2 Trending Hashtags ---
1. #ai (3 posts)
2. #python (2 posts)
```

Task #18: Smart University Hostel Management

Scenario:

A university wants to build a hostel management system that manages:

1. Room Allotment
2. Medical Emergency Handling
3. Student Record Retrieval
4. Hostel Block Connectivity
5. Visitor Entry Log

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```

C:\Users\ADMIN> OneDrive\Documents\Desktop\AIAC > lab11Q18.py > ...
69 class HostelMedicalSystem:
69     def dispatch_aid(self):
91         if not self.emergency_heap:
92             print("No active medical emergencies. All students safe.")
93             return None
94         priority, name, room = heapq.heappop(self.emergency_heap)
95         print(f"\n[DISPATCHING MEDICAL AID] -> Student: {name}, Room: {room}")
96         print(f"Reason: Priority Level {priority} condition.")
97         return name
98     def view_emergency_list(self):
99         """Displays all pending medical cases sorted by urgency."""
100         if not self.emergency_heap:
101             print("The emergency queue is currently empty.")
102         else:
103             # Sort a copy to display in priority order without emptying the heap
104             sorted_emergencies = sorted(self.emergency_heap)
105             print("\n--- Current Emergency Dashboard ---")
106             for p, name, room in sorted_emergencies:
107                 status = {1: "CRITICAL", 2: "SERIOUS", 3: "MINOR"}.get(p)
108                 print(f"[{status}] Student: {name} | Room: {room}")
109             print("-----\n")
110
111 # --- Simulation ---

```

Output :

```

PS C:\Users\ADMIN> & "C:/Program Files/Python311/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/AIAC/lab11Q18.py
Successfully registered: Priya Sharma in Room A-204
Successfully registered: Liam Smith in Room C-305

--- Hostel Record: STU002 ---
Name:    Priya Sharma
Room:    A-204
Contact: +91-87654-32109
-----

Error: Student ID STU999 not found in the registry.
Student Liam Smith has checked out and record is cleared.

```

Task #19: Smart Electricity Distribution System

Scenario:

An electricity board plans to develop a system that includes:

1. Complaint Handling
2. Fault Priority Handling
3. Consumer Record Lookup
4. Power Grid Connectivity
5. Meter Reading History Tracking

Student Task

- For each feature, select the most appropriate data structure from the list below:
 - o Stack
 - o Queue
 - o Priority Queue
 - o Linked List
 - o Binary Search Tree (BST)
 - o Graph
 - o Hash Table
 - o Deque
- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

```
C:\Users\ADMIN> OneDrive > Documents > Desktop > AIAC > lab11Q19.py > ...
1  #Write a python program that manages electricity grid faults based on urgency and impact.
2  import heapq
3  class GridFaultManager:
4      """
5      Manages electricity grid faults based on urgency and impact.
6      Priority 1: Emergency (Grid Collapse / Safety Hazard)
7      Priority 2: High Impact (Whole Neighborhood Outage)
8      Priority 3: Low Impact (Individual House Fault)
9      """
10     def __init__(self):
11         # Min-heap stores tuples: (priority_level, fault_id, location)
12         self.fault_queue = []
13
14     def report_fault(self, fault_id, location, priority):
15         """
16         Adds a new fault to the management system.
17         :param fault_id: Unique tracking ID for the fault.
18         :param location: The area or substation affected.
19         :param priority: Integer (1=Critical, 3=Minor).
20         """
21         heapq.heappush(self.fault_queue, (priority, fault_id, location))
22         severity = {1: "CRITICAL", 2: "URGENT", 3: "ROUTINE"}.get(priority, "NORMAL")
23         print(f"Logged: {fault_id} at {location} | Severity: {severity}")
```

Output :

```
PS C:\Users\ADMIN> & "C:/Program Files/Python313/python.exe" c:/Users/ADMIN/OneDrive/Documents/Desktop/AIAC/lab11Q19.py
Logged: F-101 at Baker Street | Severity: ROUTINE
Logged: F-505 at City Hospital Substation | Severity: CRITICAL
Logged: F-202 at Industrial Zone | Severity: URGENT

--- Active Grid Faults (Sorted by Priority) ---
Level 1 | ID: F-505 | Location: City Hospital Substation
Level 2 | ID: F-202 | Location: Industrial Zone
Level 3 | ID: F-101 | Location: Baker Street
```